

AUTHOR MANUSCRIPT

A Survey of Open-Source Power System Dynamic Simulators with Grid-Forming Inverter for Machine Learning Applications

Please cite the published article

Tong Su, Jiangkai Peng, Alaa Selim, Junbo Zhao, Jin Tan. A Survey of Open-Source Power System Dynamic Simulators with Grid-Forming Inverter for Machine Learning Applications. 2025 IEEE Power & Energy Society General Meeting (PESGM), 1-5 (2025).

<https://doi.org/10.1109/pesgm52009.2025.11225483>

Version and reuse

The author manuscript follows this citation sheet. Publisher pagination and formatting may differ. This sheet supplies the publication reference and reuse notice; the manuscript pages are unchanged.

© 2025 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

Publisher sharing policy

A Survey of Open-Source Power System Dynamic Simulators with Grid-Forming Inverter for Machine Learning Applications

Tong Su, *Student Member, IEEE*, Jiangkai Peng, *Member, IEEE*, Alaa Selim, *Student Member, IEEE*, Junbo Zhao, *Senior Member, IEEE*, Jin Tan, *Senior Member, IEEE*

Abstract—The emergence of grid-forming (GFM) inverter technology and the increasing role of machine learning in power systems highlight the need for evaluating the latest dynamic simulators. Open-source simulators offer distinct advantages in this field, being both free and highly customizable, which makes them well-suited for scientific research and validation of the latest models and methods. This paper provides a comprehensive survey and comparison of the latest open-source simulators that support GFM, with a focus on their capabilities and performance in machine-learning applications.

Index Terms—Open-source power system dynamic simulator, grid-forming inverter, machine learning application.

I. INTRODUCTION

THE effective implementation of machine learning models, such as deep learning (DL) and reinforcement learning (RL) relies heavily on the availability and quality of the data. As machine learning methods gain prominence in power system studies, the role of data has become increasingly critical. Given the challenges of obtaining real-world power system data, most research relies on data generated by power system simulators. This is especially true for time-domain simulations (TDSs), as existing power grids operate far from the stability boundaries, making collecting sufficient data on instability events challenging. In addition, there is always a risk of perturbations or faults occurring at various locations for large power systems. Dynamic simulators can provide a detailed and interactive power system model, enabling engineers to anticipate and analyze various fault scenarios and system responses. This includes simulations ranging from simple single-device faults to complex interactions involving multiple devices. Through these simulations, the future safety state of the system and the effectiveness of control strategies can be assessed [1]. Machine learning methods require a large amount of data from power system simulators to accurately model power system dynamics and implement control following perturbations [2], [3].

This work was authored by the National Renewable Energy Laboratory, operated by Alliance for Sustainable Energy, LLC, for the U.S. Department of Energy (DOE) under Contract No. DE-AC36-08GO28308. This material is based upon work supported by the U.S. Department of Energy's Office of Electricity under the OPEN COG Grid project.

Tong Su, Alaa Selim, and Junbo Zhao are with the Department of Electrical and Computer Engineering, University of Connecticut, Storrs, CT 06269, USA (e-mail: tongsu@uconn.edu; alaa.selim@uconn.edu; junbo@uconn.edu).

Jiangkai Peng and Jin Tan are with National Renewable Energy Laboratory, Golden, CO 80401, USA (e-mail: jiangkai.peng@nrel.gov; jin.tan@nrel.gov).

Power system simulators can be categorized into commercial and open-source simulators. Commercial simulators typically require a license but offer comprehensive user documentation, extensive functionality, highly specialized tools, as well as frequent software updates and feature upgrades. However, the advantage of open-source simulators is that they are usually free and highly customizable. Users can modify and improve the code according to specific needs, and new methods and technologies may be adopted more quickly. For example, ANDES is significantly easier to use for developing differential-algebraic equation (DAE)-based power system dynamic models compared to most commercial simulators, while maintaining high numerical efficiency [4]. Considering the ease of access and flexible scalability, open-source dynamic simulators are essential for academic research.

The existing power system simulator includes functionalities, such as power flow (PF), continuation power flow (CPF), optimal power flow (OPF), small-signal stability analysis (SSSA), and TDS. PF, CPF, and OPF are primarily based on static models to analyze the steady-state behavior under specific load and generation conditions. By contrast, SSSA and TDS utilize dynamic models to evaluate the system's dynamic response and stability following perturbations. Static models-based simulators have been well-developed, like MATPOWER [5], Pandapower [6], PowerSimulations.jl [7], PyPSA [8], PowerModels [9], PowerModelsDistribution.jl [10], OpenDSS [11] and GridLAB-D [12]. Among these, PowerModelsDistribution.jl, OpenDSS and GridLAB-D are primarily used for distribution systems. Due to the evolution of dynamic model structures and control methods, along with the level of modeling details required to capture specific phenomena of interest, there are variations in the support for dynamic models across different simulators. [13]–[15] have conducted comprehensive surveys of various power system dynamic simulators, such as power system toolbox (PST) [16] and power system analysis toolbox (PSAT) [15]. However, with the evolution of power systems and the emergence of new models, such as those for inverter-based resources (IBRs), there is a need to conduct surveys on the latest dynamic simulators in recent years.

This paper first introduces the definitions of TDSs, IBRs, and machine learning applications. It then surveys five leading open-source power system dynamic simulators that support grid-forming (GFM) inverters. Numerical studies are conducted on the IEEE 14-bus system to compare the performance of the simulators with different GFM controls.

II. POWER SYSTEM DYNAMIC SIMULATION AND MACHINE LEARNING APPLICATIONS

A. Time-Domain Simulation

TDS is categorized into quasi-static phasor (QSP) simulation and electromagnetic transient (EMT) simulation, distinguished by the simulation speed and the level of modeling details necessary for capturing specific phenomena of interest. QSP simulations represent the dynamics of transmission system circuits as discrete changes between steady-state operating points and are used for studying low-frequency phenomena, ranging from inertial response to frequency regulation. EMT simulations incorporate sufficient detail to capture fast dynamic phenomena, enabling the analysis of line dynamics, converter switching, machine fluxes, and lightning surges [1]. Furthermore, QSP simulation is divided into the positive sequence, root mean square (RMS) unbalanced, dq0-model with algebraic network, and RMS dynamic phasors. On the other hand, EMT is categorized into dq0-model, dynamic Phasors, and waveform simulation (ignores or includes switching model). The classification of TDS is shown in Fig. 1.

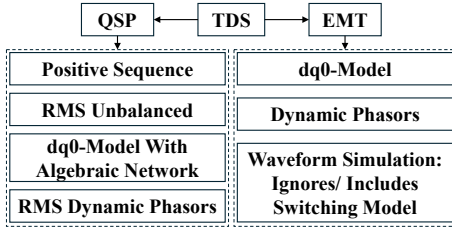


Fig. 1. Classification of power system TDS.

B. Inverter-Based Resource

Traditional inverters use grid-following (GFL) control, relying on the phase-locked loop (PLL) to estimate and track the angle of the terminal voltage and regulate the injected current into the grid. However, GFL inverters, while unable to actively control voltage and frequency, behave similarly to a current source. When facing a contingency, GFL will act with a delay and cannot mimic the instantaneous response of synchronous generators (SGs), which leads to poorer system response performance [17]. On the other hand, GFM can directly control their voltage and frequency output to inherently enhance system stability and respond to load variations [18]. The GFM source acts as a controllable voltage source and does not need the stiff terminal voltage assumption made in GFL [19]. In emergencies like grid failure or islanded mode, GFM inverters establish the grid voltage, which then serves as the reference voltage for the rest of the system.

Several GFM controls have been proposed in recent years. Droop control, which mimics the speed droop mechanism present in SGs is widely used. Then, virtual synchronous machine (VSM) control emulates the dynamics and control of SGs and can provide inertia. Matching control structurally matches the differential equations of an inverter to those of an SG. However, virtual oscillator control (VOC) mimics the synchronizing behavior of Liénard-type oscillators and can

globally synchronize an IBR-based power system. Furthermore, dispatchable VOC (dVOC) has been proposed to address the issue that the nominal power injection of VOC cannot be specified [17]. The structures of the most widely used droop control and VSM control are shown in Fig. 2 [20].

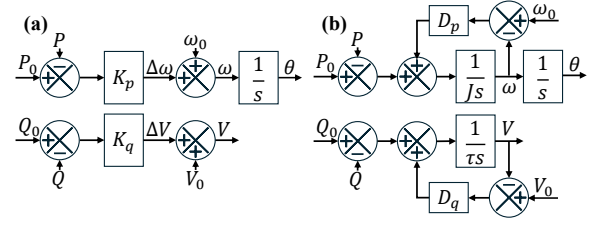


Fig. 2. Structure of the GFM control: (a) Droop control. (b) VSM control.

C. Machine Learning Application

Generally, the applications of machine learning in power systems can be divided into two categories: one focuses on forecasting, prediction, estimation, and surrogate modeling, primarily using DL; the other involves optimization, regulation, and control based on RL [2], [3]. DL methods typically require a large amount of data to accurately fit the dynamic trajectory of power systems following perturbations. However, since power systems operate with significant safety margins, dynamic data post-fault is difficult to obtain, and most existing research relies on data generated by power system simulators. To generate data, random or distribution-based Monte Carlo sampling of power system variables is necessary to cover a wide range of possible operating scenarios. These power system variables include SG and IBR active power outputs, SG and IBR bus voltage magnitudes, active and reactive power demands of loads, IBR control modes and parameters, as well as fault or perturbation locations and types, etc. RL are traditionally categorized into four types: model-based, model-free, value iteration, and policy iteration RL [3]. When considering constraints, safe RL has been proposed to maximize rewards while ensuring constraint satisfaction through a series of specially designed techniques [21]. DL depends on dynamic simulators to perform TDSs for various scenarios and perturbations, generating large amounts of simulation data to train machine learning models. In contrast, RL requires extensive interaction with the environment, performing actions, receiving rewards and states, and iteratively training the model. The environment here can be a real-world system, a dynamic simulator, or a DL-based surrogate model. The interaction between DL, RL, and simulators is shown in Fig. 3.

Therefore, when applied to machine learning tasks, dynamic simulators must be flexible enough to allow users to easily modify all parameters through code, perform simulations, read results, and operate quickly. Typically, the time required to generate data is closely related to several factors including computer performance, the scale of the system, simulation duration, simulation step size, DAEs solver algorithm, programming language, complexity of the dynamic models, and the type of TDSs. For instance, QSP-based TDSs are significantly faster than EMT-based TDSs, and multi-GPU parallel processing is typically faster than single-GPU processing.

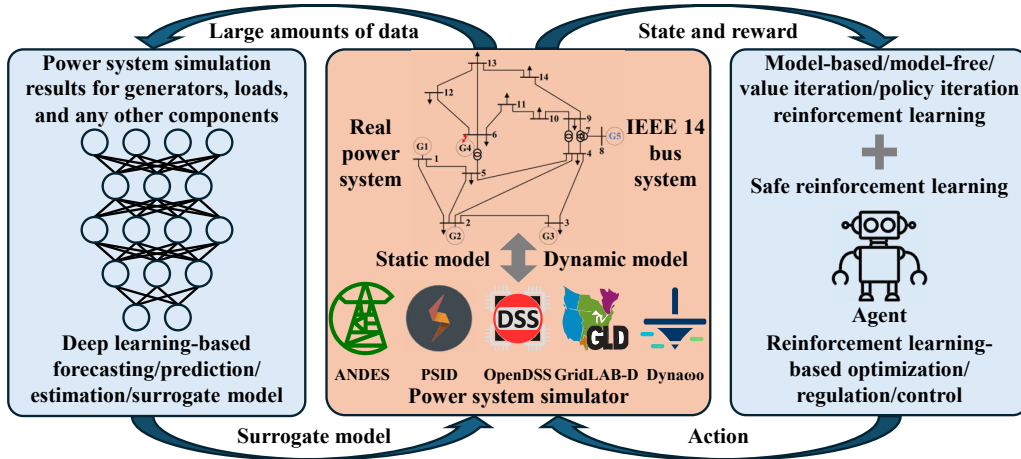


Fig. 3. Framework of interaction between DL, RL, and dynamic simulators.

III. POWER SYSTEM DYNAMIC SIMULATOR

A. ANDES

ANDES is an open-source Python library for power system modeling, computation, and analysis, and it supports PF, TDS (QSP), and SSSA for transmission systems. ANDES implements a hybrid symbolic-numeric framework for rapid prototyping of DAE-based models, including the full second-generation renewable models: REGCA1 (Renewable energy generator model type A), REGCP1 (Renewable energy generator model (REGC_A) with PLL), REGCV1 (Voltage-controlled voltage-source converter (VSC) with double-loop PI control and VSM control), REGCV2 (Voltage-controlled VSC with lag transfer functions-based inner-loop current PI controllers and VSM control), REGF1 (GFM-droop), REGF2 (GFM-VSM), and REGF3 (GFM-dVOC) [4].

B. PowerSimulationsDynamics.jl

PowerSimulationsDynamics.jl (PSID.jl) is a Julia package designed for dynamic modeling of power systems with low inertia renewable energy sources (RESs) and features an extensive library of SG, inverter, and load models. It aims to provide a flexible modeling framework to accommodate various device models based on specific needs, streamline the construction of large-scale DAEs to enhance efficiency and reduce redundancy, and leverage Julia's computational capabilities to boost performance in large-scale simulations. The supported TDS includes positive sequence QSP and dq0-model EMT [22]. The supported inverters include REGCA1, GFM-Droop, GFM-VSM, and GFM-dVOC [24].

C. Dynaωo

Dynaωo has five submodules: DynaFlow for steady-state calculations, DySym for short-circuit calculations, DynaWaltz for long-term stability simulations, DynaSwing for short-term stability studies, and DynaWave for stability studies and system design with a high penetration of power electronics-based components (quasi-EMT). The supported inverters include WECC PV, WTG4A, WTG4B, GridFormingConverterMatchingControl (GFM-Matching), GridFormingConverter-

DroopControl (GFM-Droop) and GridFormingConverterDispatchableVirtualOscillatorControl (GFM-dVOC) [23].

D. OpenDSS

OpenDSS is a powerful open-source tool for dynamic simulations in unbalanced distribution systems, widely used for high-penetration DERs, restoration actions, and GFM control. It supports three dynamic modes: (1) Default mode using built-in generator models (e.g., model=1 for second-order SGs or model=7 for IBRs), (2) DynamicExp function (available in OpenDSS v9) to define custom DAEs for advanced controls, and (3) User-written DLL files for flexible modeling SGs and GFMs. These DLL files can be created in either the Delphi programming language or Free Pascal [25], allowing detailed modeling and control of internal structures and dynamic state variables. OpenDSS also includes a COM interface, enhancing integration with other software platforms. Configurations are specified in OpenDSS scripts with support for CSV and COMTRADE formats, and OpenDSS's API compatibility with Python and MATLAB enables further customization [11].

E. GridLAB-D

GridLAB-D is an open-source tool tailored for dynamic simulations in distribution systems, supporting TDS of QSP models. It enables analysis of time-dependent behaviors such as load variability, demand response, and distributed energy resource integration, with basic models for PV, battery, wind systems, and GFM with droop control for stability studies. GridLAB-D utilizes the GLM (GridLAB-D Model) format for model configurations and supports XML for data exchange, ensuring compatibility with external tools. Additionally, its Python and C/C++ API integration allows for extended dynamic simulation and custom module development [12].

We conducted a comprehensive comparison of these simulators based on two main aspects: (1) Base functions and model, including programming language, support for unbalanced modeling (UM), PF, SSSA, type of TDS, RES models, and parallel computing capabilities, as shown in Table I; (2) Machine learning applications, focusing on whether they

TABLE I
COMPARISON OF DYNAMIC SIMULATORS: BASE FUNCTIONS AND MODELS

Simulator	Language	UM	PF	SSSA	TDS	RES model	Parallel
ANDES [4]	Python	✗	✓	✓	QSP (positive sequence)	REGCA1, REGCP1, REGCV1, REGCV2, GFM-Droop, GFM-VSM, GFM-dVOC	✓
PSID.jl [22]	Julia	✗	✓	✓	QSP (positive sequence)/ EMT (dq0-model)	REGCA1, GFM-Droop, GFM-VSM, GFM-dVOC	/
Dynaω [23]	C++/Modelica	✗	✓	✓	QSP and quasi-EMT	WECC PV, WTG4A, WTG4B, GFM-Matching, GFM-Droop, GFM-dVOC	/
OpenDSS [11]	Delphi	✓	✓	✗	QSP (positive sequence)	GFM-Droop, GFM-VSM, PV models, storage models, customized models (.dll)	/
GridLAB-D [12]	C/C++	✓	✓	✗	QSP (positive sequence)	Basic PV, Battery, Wind, GFM-Droop (limited)	✓

✓, ✗ and / represent that the simulator has, does not have, and does not mention this function, respectively.

TABLE II
COMPARISON OF DYNAMIC SIMULATORS: MACHINE LEARNING APPLICATIONS

Simulator	Manual	Interaction with external code	Supported external data format	Simulation time (1/1000 samples)
ANDES	Detailed descriptions of all functions, models, and parameters	Easily implement Python-based parameter modification and result retrieval	MATPOWER, PSS/E, JSON	1.27s/224s
PSID.jl	Detailed descriptions of all functions, models, and parameters	Easily implement Julia-based parameter modification and result retrieval	MATPOWER, PSS/E, JSON	0.05s/31.1s
Dynaω	Brief overview without descriptions of models or parameters	Executed via terminal command line, with limited code-based interaction	Unknown	0.34s/374s
OpenDSS	Comprehensive documentation on functions and models	Supports interaction with external scripts in Python and MATLAB for parameter modification and data retrieval	CSV, COMTRADE	/
GridLAB-D	Detailed documentation with examples for functions and models	Supports integration with Python and C/C++ for extensive customization and external module development	CSV, XML, JSON	/

The simulation times for OpenDSS and GridLAB-D are not applicable (indicated as "/") because they are primarily designed for distribution networks.

provide detailed manuals, allow flexible code-based parameter modification and result retrieval, supported external data formats, and their simulation time (IEEE 14-bus system; step size 0.01s; simulation duration 10s), as shown in Table II. It should be noted that, due to Python's slower speed compared to Julia and C++, ANDES has the longest simulation time for a single run. However, with multi-CPU parallelization, the simulation time for 1,000 samples remains competitive.

IV. SIMULATION COMPARISON AND CASE STUDIES

Since ANDES and PSID.jl provide detailed manuals and programming guidance, TDSs were performed on these two simulators using a modified IEEE 14-bus system, where generators 1-4 are SGs and generator 5 is set as SG, GFM-Droop, and GFM-VSM sequentially. The fault is configured as a generation trip of generator 4, beginning at 1s, with a total simulation duration of 10s. The modified 14-bus system structure, ANDES simulation results, and PSID.jl simulation results are shown in Fig. 3, 4, and 5, respectively. The simulation results of the two simulators are very similar, with differences arising from variations in dynamic models and control parameters.

V. CONCLUSION

This paper presents an in-depth comparison of the latest open-source dynamic simulators that support GFM inverters, with a focus on their suitability for machine learning applications. This survey offers machine learning professionals

practical guidance on power system data generation and interaction with dynamic simulators. Future developments include integrating new models, developing new parallel methods, simplifying integration processes, handling false data, and advancing cyber-physical co-simulation techniques [26].

ACKNOWLEDGMENTS

The views expressed in the article do not necessarily represent the views of the DOE or the U.S. Government. The U.S. Government retains, and the publisher, by accepting the article for publication, acknowledges that the U.S. Government retains a nonexclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this work or allow others to do so for U.S. Government purposes.

REFERENCES

- [1] J. D. Lara, R. Henriquez-Auba, D. Ramasubramanian, S. Dhople, D. S. Callaway, and S. Sanders, "Revisiting power systems time-domain simulation methods and models," *IEEE Trans. Power Syst.*, vol. 39, no. 2, pp. 2421–2437, Mar. 2024.
- [2] M. S. Ibrahim, W. Dong, and Q. Yang, "Machine learning driven smart electric power systems: Current trends and new perspectives," *Appl. Energy*, vol. 272, 2020, Art. no. 115237.
- [3] X. Chen, G. Qu, Y. Tang, S. Low, and N. Li, "Reinforcement learning for selective key applications in power systems: Recent advances and future challenges," *IEEE Trans. Smart Grid*, vol. 13, no. 4, pp. 2935–2958, Jul. 2022.
- [4] H. Cui, F. Li, and K. Tomsovic, "Hybrid symbolic-numeric framework for power system modeling and analysis," *IEEE Trans. Power Syst.*, vol. 36, no. 2, pp. 1373–1384, Mar. 2021.

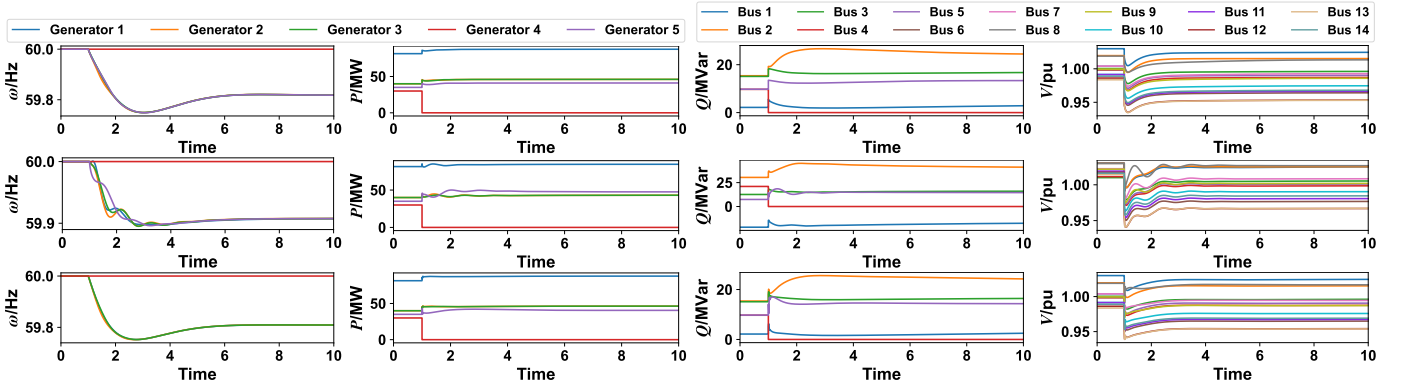


Fig. 4. ANDES simulation results: from top to bottom, the three rows correspond to G5 configured as SG, GFM-Droop, and GFM-VSM, respectively; From left to right are the speed ω , active power generation P , and reactive power generation Q of the SG or GFM, followed by the voltage V of all buses.

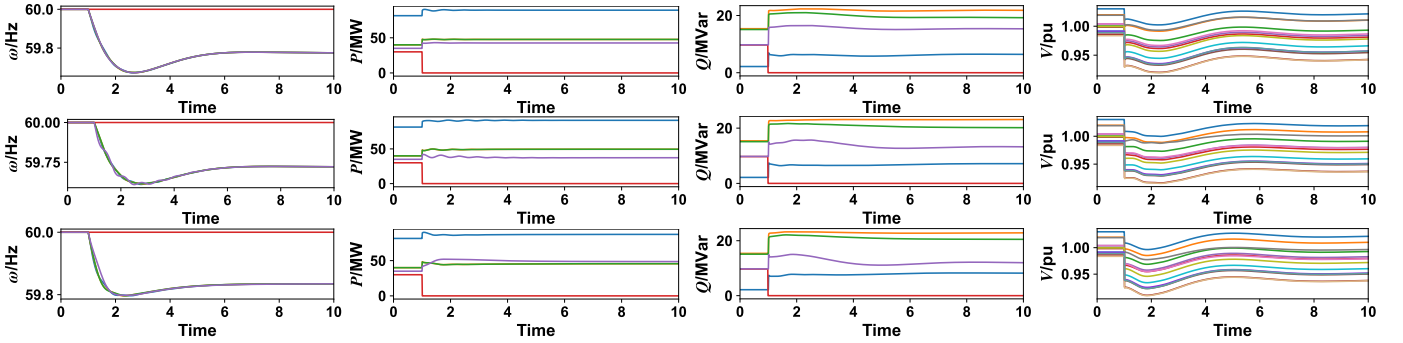


Fig. 5. PSID.jl simulation results: from top to bottom, the three rows correspond to G5 configured as SG, GFM-Droop, and GFM-VSM, respectively; From left to right are the speed ω , active power generation P , and reactive power generation Q of the SG or GFM, followed by the voltage V of all buses.

- [5] R. D. Zimmerman, C. E. Murillo-Sánchez, and R. J. Thomas, "MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education," *IEEE Trans. Power Syst.*, vol. 26, no. 1, pp. 12–19, Feb. 2011.
- [6] L. Thurner, A. Scheidler, F. Schäfer, J.-H. Menke, J. Dolichon, F. Meier, S. Meinecke, and M. Braun, "Pandapower—an open-source Python tool for convenient modeling, analysis, and optimization of electric power systems," *IEEE Trans. Power Syst.*, vol. 33, no. 6, pp. 6510–6521, Nov. 2018.
- [7] J. D. Lara, C. Barrows, D. Thom, D. Krishnamurthy, and D. Callaway, "PowerSystems.jl—a power system data management package for large scale modeling," *SoftwareX*, vol. 15, 2021, Art. no. 100747.
- [8] T. Brown, J. Hörsch, and D. Schlachtberger, "PyPSA: Python for power system analysis," *J. Open Res. Softw.*, vol. 6, no. 4, Jan. 2018.
- [9] C. Coffrin, R. Bent, K. Sundar, Y. Ng, and M. Lubin, "PowerModels.jl: An open-source framework for exploring power flow formulations," in *Proc. Power Syst. Comput. Conf.*, Jun. 2018, pp. 1–8.
- [10] D. M. Fobes, S. Claeys, F. Geth, and C. Coffrin, "PowerModelsDistribution.jl: An open-source framework for exploring distribution power flow formulations," *Electric Power Syst. Res.*, vol. 189, 2020, Art. no. 106664.
- [11] R. C. Dugan and T. E. McDermott, "An open source platform for collaborating on smart grid research," in *Proc. IEEE Power Energy Soc. General Meeting*, 2011, pp. 1–7.
- [12] D. P. Chassin, J. C. Fuller, and N. Djilali, "GridLAB-D: an agent-based simulation framework for smart grids," *J. Appl. Math.*, vol. 2014, no. 1, 2014, Art. no. 492320.
- [13] L. Bam and W. Jewell, "Review: Power system analysis software tools," in *Proc. IEEE Power Energy Soc. General Meeting*, 2005, pp. 139–144.
- [14] M. Vogt, F. Marten, and M. Braun, "A survey and statistical analysis of smart grid co-simulations," *Appl. Energy*, vol. 222, pp. 67–78, 2018.
- [15] F. Milano, "An open source power system analysis toolbox," *IEEE Trans. Power Syst.*, vol. 20, no. 3, pp. 1199–1206, Aug. 2005.
- [16] J. H. Chow and K. W. Cheung, "A toolbox for power system dynamics and control engineering education and research," *IEEE Trans. Power Syst.*, vol. 7, no. 4, pp. 1559–1564, Nov. 1992.
- [17] A. Tayyebi, D. Groß, A. Anta, F. Kupzog, and F. Dörfler, "Frequency stability of synchronous machines and grid-forming power converters," *IEEE J. Emerg. Sel. Topics Power Electron.*, vol. 8, no. 2, pp. 1004–1018, 2020.
- [18] D. Pattabiraman, R. Lasseter, and T. Jahns, "Comparison of grid following and grid forming control for a high inverter penetration power system," in *Proc. IEEE Power Energy Soc. Gen. Meeting*, 2018, pp. 1–5.
- [19] J. Matevosyan, B. Badrzadeh, T. Prevost, E. Quitmann, D. Ramasubramanian, H. Urdal, S. Achilles, J. MacDowell, S. H. Huang, V. Vital *et al.*, "Grid-forming inverters: Are they the key for high renewable penetration?" *IEEE Power Energy Mag.*, vol. 17, no. 6, pp. 89–98, 2019.
- [20] D. Pan, X. Wang, F. Liu, and R. Shi, "Transient stability of voltage-source converters with grid-forming control: A design-oriented study," *IEEE J. Emerg. Sel. Topics Power Electron.*, vol. 8, no. 2, pp. 1019–1033, 2019.
- [21] T. Su, T. Wu, J. Zhao, A. Scaglione, and L. Xie, "A review of safe reinforcement learning methods for modern power systems," *arXiv preprint arXiv:2407.00304*, 2024.
- [22] J. D. Lara, R. Henriquez-Auba, M. Bossart, D. S. Callaway, and C. Barrows, "PowerSimulationsDynamics.jl—an open source modeling package for modern power systems with inverter-based resources," *arXiv preprint arXiv:2308.02921*, 2023.
- [23] A. Guirionnet, M. Saugier, S. Petitrenaud, F. Xavier, and P. Panciatici, "Towards an open-source solution using modelica for time-domain simulation of power systems," in *Proc. IEEE PES Innovative Smart Grid Technol. Conf. Europe*, 2018, pp. 1–6.
- [24] R. Henriquez-Auba, J. D. Lara, and D. S. Callaway, "Small-signal stability impacts of load and network dynamics on grid-forming inverters," in *Proc. IEEE Power Energy Soc. Innov. Smart Grid Technol. Conf.*, 2024, pp. 1–5.
- [25] M. Abiola-Ellison, *Getting Started with Lazarus and Free Pascal: a beginners and intermediate guide to Free Pascal using Lazarus IDE*. CreateSpace Independent Publishing Platform, 2015.
- [26] D. Qiu, M. Liu, and X. Zhang, "A power-communication co-simulation platform for real-time microgrid cyber security analysis," in *Proc. Int. Conf. Artif. Intell. Things Syst.* IEEE, 2024, pp. 1–6.